Solution Code - Module 3-Core Java - TCPClientServerChat

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: Q35_TCPClientServerChat.java

```
 1 | import java.io.BufferedReader;
 2 | import java.io.InputStreamReader;
 3 | import java.io.PrintWriter;
 4 | import java.net.ServerSocket;
 5 | import java.net.Socket;
 6 | import java.util.Scanner;
 7 |
 8 | public class Q35_TCPClientServerChat {
 9 |     private static final int DEFAULT_PORT = 5000;
10 |
11 |     public static void main(String[] args) {
12 |         if (args.length > 0) {
13 |             String role = args[0].toLowerCase();
14 |             if (role.equals("server")) {
15 |                 runServer(DEFAULT_PORT);
16 |             } else if (role.equals("client")) {
17 |                 runClient("localhost", DEFAULT_PORT);
18 |             } else {
19 |                 System.out.println("Unknown role. Usage: java Q35_TCPClientServerChat [server|client]");
20 |             }
21 |         } else {
22 |             // Self-running demonstration mode
23 |             System.out.println("=== TCP Client-Server Chat Simulation ===");
24 |             System.out.println("No arguments provided. Running automated thread-based simulation.");
25 |             runSimulation();
26 |         }
27 |     }
28 |
29 |     private static void runServer(int port) {
30 |         System.out.println("[Server] Starting server on port " + port + "...");
31 |         try (ServerSocket serverSocket = new ServerSocket(port)) {
32 |             System.out.println("[Server] Waiting for client to connect...");
33 |             try (Socket clientSocket = serverSocket.accept();
   34   |                                                  BufferedReader   in   =   new   BufferedReader(new
InputStreamReader(clientSocket.getInputStream()));
35 |                  PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
36 |                  Scanner scanner = new Scanner(System.in)) {
37 |
   38   |                                              System.out.println("[Server]   Client   connected:   "   +
clientSocket.getRemoteSocketAddress());
39 |                 out.println("Welcome to the TCP Chat Server! Type 'bye' to exit.");
40 |
41 |                 String clientMsg;
```

Solution Code - Module 3-Core Java - TCPClientServerChat

```
                while ((clientMsg = in.readLine()) != null) {
                    System.out.println("[Client says]: " + clientMsg);
                    if (clientMsg.equalsIgnoreCase("bye")) {
                        out.println("Goodbye client!");
                        break;
                    }
                    // Interactive mode (reading from console input for server)
                    System.out.print("[Server reply]: ");
                    if (scanner.hasNextLine()) {
                        String reply = scanner.nextLine();
                        out.println(reply);
                        if (reply.equalsIgnoreCase("bye")) {
                            break;
                        }
                    } else {
                        out.println("Received: " + clientMsg);
                    }
                }
            }
        } catch (Exception e) {
            System.err.println("[Server Error]: " + e.getMessage());
        }
    }

    private static void runClient(String host, int port) {
        System.out.println("[Client] Connecting to " + host + ":" + port + "...");
        try (Socket socket = new Socket(host, port);
             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
             Scanner scanner = new Scanner(System.in)) {

            System.out.println("[Client] Connected successfully!");
            // Read welcome message
            System.out.println("[Server says]: " + in.readLine());

            while (true) {
                System.out.print("[Client send]: ");
                if (scanner.hasNextLine()) {
                    String clientMsg = scanner.nextLine();
                    out.println(clientMsg);
                    if (clientMsg.equalsIgnoreCase("bye")) {
                        System.out.println("[Client] Connection closed.");
                        break;
                    }
                    String serverReply = in.readLine();
                    if (serverReply == null) {
                        System.out.println("[Client] Server disconnected.");
                        break;
                    }
                    System.out.println("[Server reply]: " + serverReply);
```

Solution Code - Module 3-Core Java - TCPClientServerChat

```
 92 |                     if (serverReply.equalsIgnoreCase("bye")) {
 93 |                         break;
 94 |                     }
 95 |                 } else {
 96 |                     break;
 97 |                 }
 98 |             }
 99 |         } catch (Exception e) {
100 |             System.err.println("[Client Error]: " + e.getMessage());
101 |         }
102 |     }
103 |
104 |     private static void runSimulation() {
105 |         final int simPort = 5055;
106 |         // Start server in background thread
107 |         Thread serverThread = new Thread(() -> {
108 |             try (ServerSocket serverSocket = new ServerSocket(simPort)) {
109 |                 try (Socket socket = serverSocket.accept();
110  |                                                     BufferedReader   in   =   new   BufferedReader(new
InputStreamReader(socket.getInputStream()));
111 |                      PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
112 |
113 |                     out.println("Hello Client! I am the automated test server.");
114 |                     String msg = in.readLine();
115 |                     System.out.println("[Sim-Server] Received: \"" + msg + "\"");
116 |                     out.println("I received your message: " + msg);
117 |
118 |                     msg = in.readLine();
119 |                     System.out.println("[Sim-Server] Received: \"" + msg + "\"");
120 |                     out.println("Goodbye!");
121 |                 }
122 |             } catch (Exception e) {
123 |                 System.err.println("[Sim-Server Error]: " + e.getMessage());
124 |             }
125 |         });
126 |
127 |         serverThread.start();
128 |
129 |         // Let the server socket start up
130 |         try {
131 |             Thread.sleep(500);
132 |         } catch (InterruptedException ignored) {}
133 |
134 |         // Connect client from main thread
135 |         try (Socket socket = new Socket("localhost", simPort);
136 |              BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
137 |              PrintWriter out = new PrintWriter(socket.getOutputStream(), true)) {
138 |
139 |             System.out.println("[Sim-Client] Connected to server.");
140 |             System.out.println("[Sim-Client] Server says: \"" + in.readLine() + "\"");
```

Solution Code - Module 3-Core Java - TCPClientServerChat

```
141 |
142 |             System.out.println("[Sim-Client] Sending: \"Ping\"");
143 |             out.println("Ping");
144 |
145 |             System.out.println("[Sim-Client] Server replied: \"" + in.readLine() + "\"");
146 |
147 |             System.out.println("[Sim-Client] Sending: \"Close Connection\"");
148 |             out.println("Close Connection");
149 |             System.out.println("[Sim-Client] Server final reply: \"" + in.readLine() + "\"");
150 |
151 |         } catch (Exception e) {
152 |             System.err.println("[Sim-Client Error]: " + e.getMessage());
153 |         }
154 |
155 |         try {
156 |             serverThread.join();
157 |         } catch (InterruptedException ignored) {}
158 |         System.out.println("[Simulation] Complete.");
159 |     }
160 | }
```